

1. Introduction to Servlets

2. Servlets are Java programs that run on a web server, handling client requests and generating dynamic responses.
3. Servlet used to process or store data submitted by an HTML form, provide dynamic content like returning results from a database query, manage state information, etc.

2. Servlet Lifecycle

The lifecycle of a servlet is controlled by the **container** (e.g., Tomcat), and it goes through the following stages:

Java Programming Course

1. **Loading and Instantiation:** The servlet class is loaded and an instance is created.
2. **Initialization (init method):** The init method is called once to initialize the servlet.
3. **Request Handling (service method):** The service method is called for each request to process it.
4. **Destruction (destroy method):** The destroy method is called once before the servlet is removed from service.

1. Setting Up the Environment

Required Tools

- **JDK:** Java Development Kit
- **Apache Tomcat:** Web server and servlet container
- **Eclipse/IntelliJ IDEA:** Integrated Development Environment (IDE)
- Configure tomcat with IDE

2. Creating a Simple Project

1. **Eclipse Setup:**
 - Open Eclipse and create a new Dynamic Web Project.
 - Set up Apache Tomcat in Eclipse.
 - Create a new servlet class.

3. Creating First Servlet

There are three ways to create servlet

1. implementing **Servlet** Interface
2. extending **GenericServlet** class

3. extending HttpServlet class.

Example:

```
import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/hello")

public class HelloServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/html");

        response.getWriter().println("<h1>Hello, World!</h1>");

    }

}
```

5. Handling Requests and Responses

They handle HTTP requests from clients ([browsers](#)) and generate dynamic responses.

1. Client Sends Request:

- A client (browser) sends an HTTP request to the web server containing information like:
 - **Request Method (GET, POST, etc.)**
 - **URL of the requested resource (Servlet)**
 - **Headers (e.g., user agent, cookies)**
 - **Optional request body (for POST requests)**

2. Servlet Container Intercepts:

- The web server's Servlet container intercepts the request.
- It identifies the corresponding Servlet based on the URL mapping defined in web.xml.

3. Servlet Invoked (doGet/doPost):

- The container invokes the appropriate method within the Servlet class:
 - `doGet(HttpServletRequest request, HttpServletResponse response)` for GET requests.

- `doPost(HttpServletRequest request, HttpServletResponse response)` for POST requests.
- Other methods exist for less common HTTP methods (PUT, DELETE).

4. Processing Request:

- Inside the `doGet` or `doPost` method, the Servlet retrieves information from the request object:
 - **Request Parameters:** Use `getParameter(String name)` to access form data or query string parameters.
 - **Headers:** Use `getHeader(String name)` to access request headers.
 - **Request Body:** Use `getInputStream()` for binary data or a `BufferedReader` for text data (for POST requests).

5. Generating Response:

- The Servlet processes the request information and generates the response content.
- It uses the `HttpServletResponse` object to control the response:
 - **Setting Content Type:** Use `setContentType(String type)` (e.g., "text/html" for HTML content).
 - **Writing Response:** Use `getWriter()` for text output or `getOutputStream()` for binary data.
 - **Setting Status Code:** Use `setStatus(int code)` to indicate success (200 OK), redirect (302 Found), or error (404 Not Found).

6. Sending Response:

- The Servlet container packages the response object and sends it back to the client (browser).

6. Creating a Feedback Form Application with Servlets

1. Define the Feedback Form (HTML):

Create an HTML file (`feedback.html`) with a form containing fields for user information (optional) and feedback message:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Feedback Form</title>
</head>
```

```
<body>

<h1>Feedback Form</h1>

<form action="submitFeedback" method="post">

  <label for="name">Name (Optional):</label>

  <input type="text" id="name" name="name"><br>

  <label for="feedback">Feedback:</label>

  <textarea id="feedback" name="feedback" rows="5" cols="30"></textarea><br>

  <button type="submit">Submit Feedback</button>

</form>
</body>
</html>
```

2. Create a Feedback Servlet (Java):

Java Programming Course

- Create a Java class (FeedbackServlet.java) extending HttpServlet.
- Override the doPost method to handle form submissions:

```
public class FeedbackServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Get feedback message from the request
        String name = request.getParameter("name"); // Optional
        String feedback = request.getParameter("feedback");

        // Process the feedback (e.g., store in database, send email)
        System.out.println("Received feedback from: " + name);
        System.out.println("Feedback message: " + feedback);

        // (Optional) Send a success response
        response.setContentType("text/html");
```

```
PrintWriter out = response.getWriter();

out.println("<h1>Thank you for your feedback!</h1>");

}

}
```

3. Web.xml Configuration:

- Create a web.xml file to map the URL pattern for the feedback form and the Servlet:

```
<web-app>

<servlet>

  <servlet-name>FeedbackServlet</servlet-name>

  <servlet-class>com.yourpackage.FeedbackServlet</servlet-class>

</servlet>

<servlet-mapping>

  <servlet-name>FeedbackServlet</servlet-name>

  <url-pattern>/submitFeedback</url-pattern>

</servlet-mapping>

</web-app>
```

7. Session Management

Session management is crucial in web applications to maintain user state across multiple requests.

1. Using HttpSession:
 - The most common approach.
 - Servlets leverage the HttpSession object to store user-specific data.

```
public class LoginServlet extends HttpServlet {

protected void doPost(HttpServletRequest request, HttpServletResponse response)

  throws ServletException, IOException {

  String username = request.getParameter("username");
```

```

String password = request.getParameter("password");

// (Assuming successful login logic)
HttpSession session = request.getSession();
session.setAttribute("loggedInUser", username);

// Redirect to a welcome page
response.sendRedirect("welcome.jsp");
}
}

public class WelcomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession();
        String username = (String) session.getAttribute("loggedInUser");

        if (username != null) {
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.println("<h1>Welcome, " + username + "!</h1>");
        } else {
            // Redirect to login page if not logged in
            response.sendRedirect("login.html");
        }
    }
}

```

2. Using Cookies:

- Stores data on the client-side (user's browser).

- Less secure than sessions for sensitive information (due to potential theft).

```
public class RememberMeServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String rememberMe = request.getParameter("rememberMe");

        if (rememberMe != null && rememberMe.equals("on")) {
            // Create a cookie to store a user identifier
            String userId = "12345"; // (Assuming user ID generation)
            Cookie cookie = new Cookie("userId", userId);
            cookie.setMaxAge(60 * 60 * 24 * 7); // Expires in a week
            response.addCookie(cookie);
        }

        // Redirect to the main page
        response.sendRedirect("index.jsp");
    }
}
```

3. URL Rewriting (Less Secure):

- Encodes session ID in the URL.
- Not recommended due to security concerns and potential for long URLs.

// (Pseudocode, not recommended for real applications)

```
String sessionId = request.getSession().getId();
String targetUrl = "welcome.jsp?sessionId=" + sessionId;
response.sendRedirect(targetUrl);
```

8. Error Handling

Configure custom error pages in web.xml.

```
<error-page>

  <error-code>404</error-code>

  <location>/error404.html</location>

</error-page>
```

9. ServletContext and ServletConfig

ServletContext

- **Definition:** ServletContext is an interface provided by the servlet container to communicate with its environment.
- **Purpose:** It allows servlets to access web application parameters, communicate with other servlets, and share resources.
- **Scope:** The ServletContext is available to all the servlets and JSPs in a web application, making it suitable for sharing global information.

Key Methods:

- **getInitParameter(String name):** Returns the value of a context-wide initialization parameter.
- **getAttribute(String name):** Retrieves an attribute stored in the context.
- **setAttribute(String name, Object object):** Stores an attribute in the context.
- **getRealPath(String path):** Returns the real path of a file on the server.

```
public class ContextExampleServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        ServletContext context = getServletContext();

        String paramValue = context.getInitParameter("globalParam");
        response.getWriter().println("Global Param: " + paramValue);

        context.setAttribute("sharedData", "This is shared across the application");
        String sharedData = (String) context.getAttribute("sharedData");
        response.getWriter().println("Shared Data: " + sharedData);
    }

}
```

ServletConfig

- **Definition:** ServletConfig is an interface that provides servlet-specific configuration information to a servlet.
- **Purpose:** It allows a servlet to access initialization parameters defined for it in the deployment descriptor (web.xml) or annotations.
- **Scope:** ServletConfig is unique to each servlet, meaning it contains initialization parameters specific to the servlet it is associated with.

Key Methods:

- **getServletName():** Returns the name of the servlet.
- **getInitParameter(String name):** Retrieves the value of a specific initialization parameter for the servlet.
- **getServletContext():** Returns the ServletContext object for the web application.

```
public class ConfigExampleServlet extends HttpServlet {
    private String initParam;

    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        initParam = config.getInitParameter("initParam");
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.getWriter().println("Init Param: " + initParam);
    }
}
```

xml configuration

```
<servlet>
    <servlet-name>ConfigExampleServlet</servlet-name>
    <servlet-class>com.example.ConfigExampleServlet</servlet-class>
    <init-param>
        <param-name>initParam</param-name>
        <param-value>This is an init parameter</param-value>
    </init-param>
</servlet>
```

```
</init-param>
</servlet>
<servlet-mapping>
  <servlet-name>ConfigExampleServlet</servlet-name>
  <url-pattern>/configExample</url-pattern>
</servlet-mapping>
```